REPORT

Object-Oriented Programming Mini-Project Report

Project title: Traditional Game – Ô ăn quan
Topic: Topic 4 — Traditional game: Ô ăn quan
Group: 9
Lecturer: TS. Nguyen Thi Thu Trang

---

1. Assignment of Members

Report in a separate sheet

---

2. Mini-Project Description

2.1 Objective

This project implements the traditional Vietnamese board game **Ô ăn quan** as an interactive desktop application using Java and JavaFX. The goal is to apply Object-Oriented Programming principles such as encapsulation, inheritance, abstraction, and polymorphism to model the game board, players, squares, rules, and gameplay mechanics.

2.2 Requirements

- The GUI can be freely designed, focus should remain on OOP design and logic
- Display the game board with 10 citizen squares and 2 mandarin squares.
- Players information need to be visible.
- Allow two players to take turns selecting a square and a direction, indicate whose turn it currently is.
- Implement stone spreading and capturing rules correctly and dynamically.
- Update player scores dynamically.
- End the game when both mandarin squares are empty.
- Announce the winner when the game ends.
- Provide a main menu, help screen, and game screen, always include a Back button to return to the menu.
- Quit the game with confirmation

- Provide dynamic interactions between components in GUI:

---

3. Use Case Diagram and Explanation

3.1 Actor(s) & Use Cases

Use Case	Actor	Description
Start Game	Player	Starts a new Ô ăn quan game session and initializes the board and players.
View Help / Rules	Player	Displays the instructions and rules of the Ô ăn quan game.
Exit Application	Player	Exits the application after confirmation from the user.
Select Square	Player	Selects a square on the board to pick up and spread stones.
Choose Direction	Player	Chooses the spreading direction (clockwise or counter-clockwise).
View Board	Player	Displays the current state of the game board and stones.
View Score	Player	Displays the current scores of both players.
End Game	Player	Ends the current game when the termination condition is reached and shows the final result.

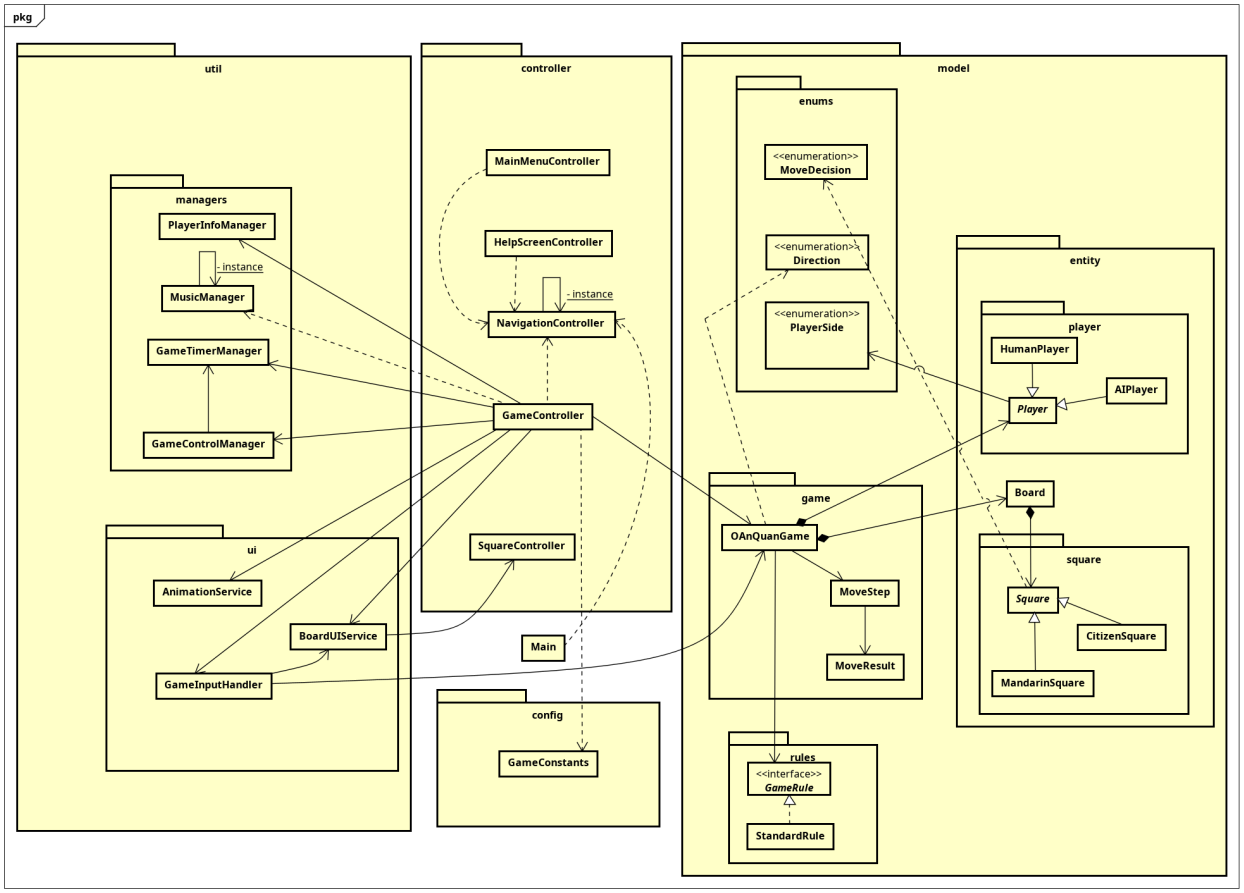
3.2 Explanation

The user interacts with the game via the GUI to select squares and directions. The controller sends these actions to the game engine, which updates the board state and returns the results to the UI for visualization.

---

4. Design

4.1 General Class Diagram (Package Level)



Packages:

- `controller` — controllers
- `model.entity` — core entities of model
- `model.game` — game engine
- `model.rules` — rule implementations
- `util` — utilities (managers, services)
- `config` — constants

Relationships:

- Model:

- `OAnQuanGame` composites `Board`, `Player`, and `GameRule`.
- `Square` is the superclass of `CitizenSquare` and `MandarinSquare`.
- `Player` is the superclass of `HumanPlayer` and `AIPlayer`.
- `Game Rule` is an interface, with `StandardRule` is the implementation
- `Board` composites `Square`
- Enums: used to provide meaningful context to the model (avoid reusing the same number everywhere)

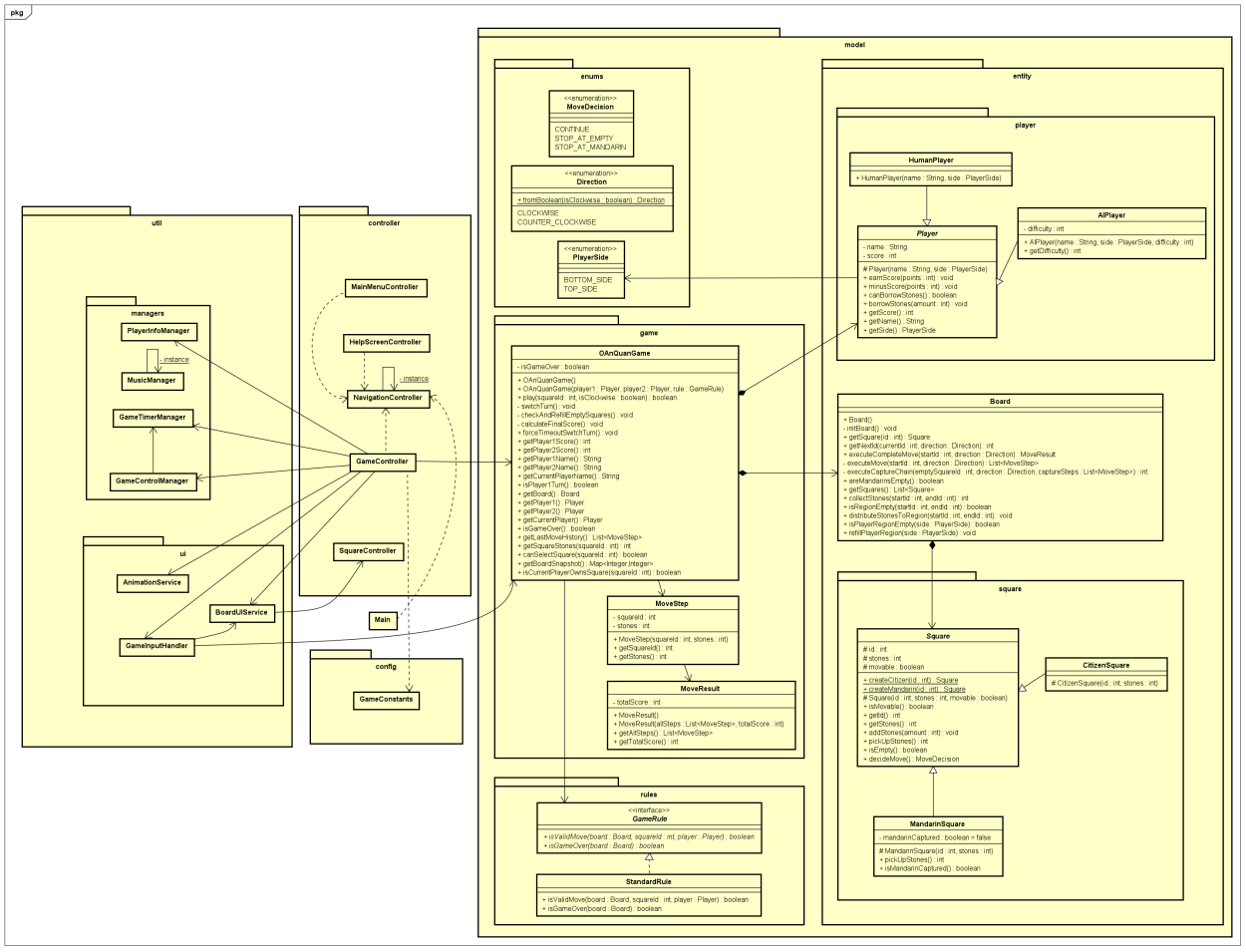
- Controllers:

- `GameController` aggregates `OAnQuanGame` and helper services
- All controllers except `NavigationController` and `SquareController` depends on it.
- `SquareController` is a part of `BoardUIService`

- Services and Managers: parts of `GameController`

4.2 Detail for Classes / Methods (Noticeable)

`Model`



`OAnQuanGame`

- `play(int squareId, boolean isClockwise)` : main method for OAnQuanGame, used to perform move logic
- `forceTimeOutSwitchTurn()` : method for timeout situation

`Board`

- `getNextId(int currentId, Direction direction)`: method for providing next index based on direction
- `collectStones(int startId, int endId)`: helper for calculating final scores

`Square`

- `decideMove()`: helper methods for validating move

`StandardRule`

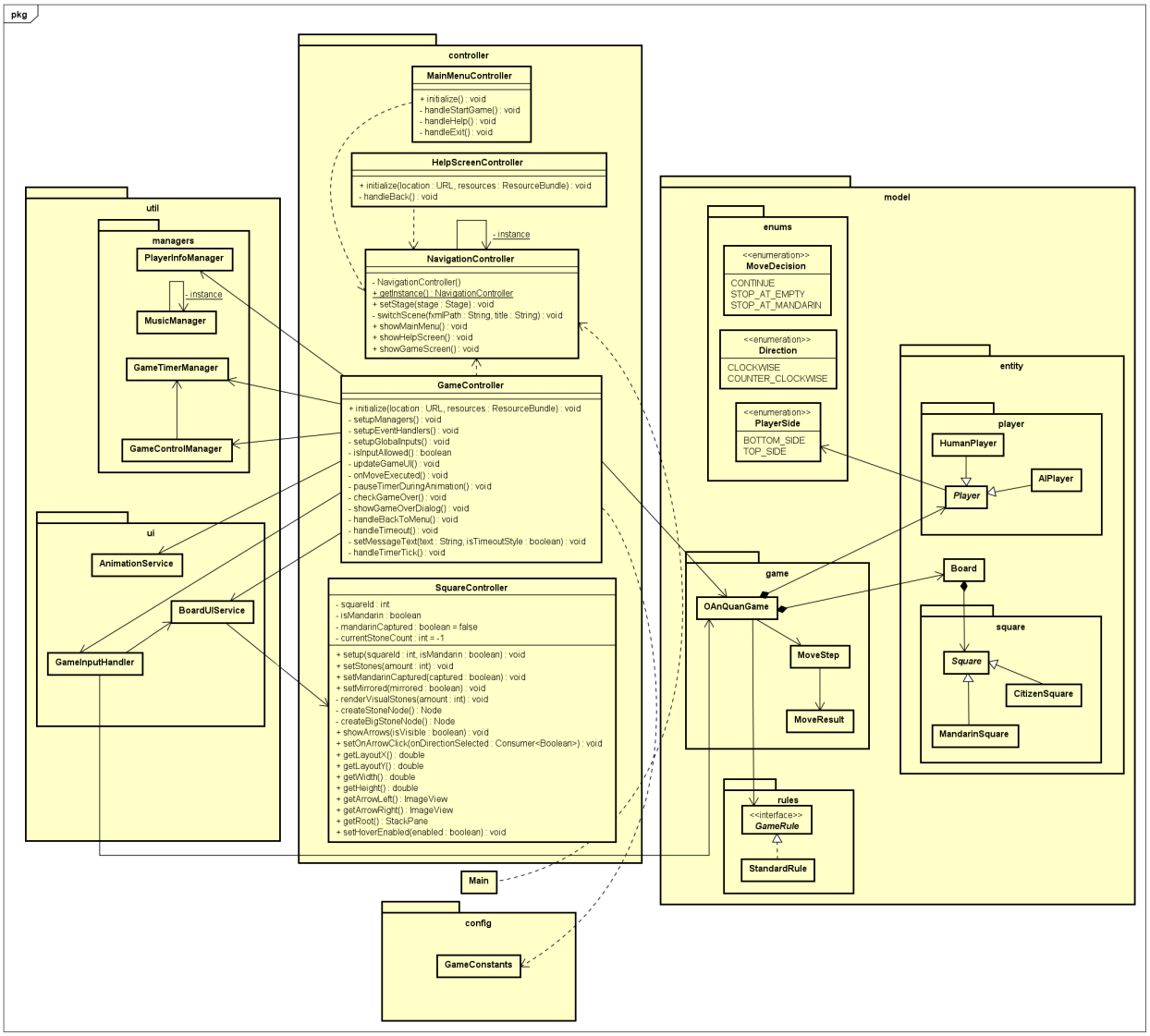
- `isValidMove(Board board, int squareId, Player player)`: validate move
- `isGameOver(Board board)`: check game over

`Player`

- `canBorrowStones()` : helper method for performing a move

`MoveResult`: RECORD CHANGES AFTER PERFORMING A MOVE

`Controller`



`GameController`

- `onMoveExecuted()` : main method for coordinating model and view
- `checkGameOver()`: method for manipulating view to show notifications about winner
- `handleTimeout()`: method for handling situation where timeout happens (skip playing move)

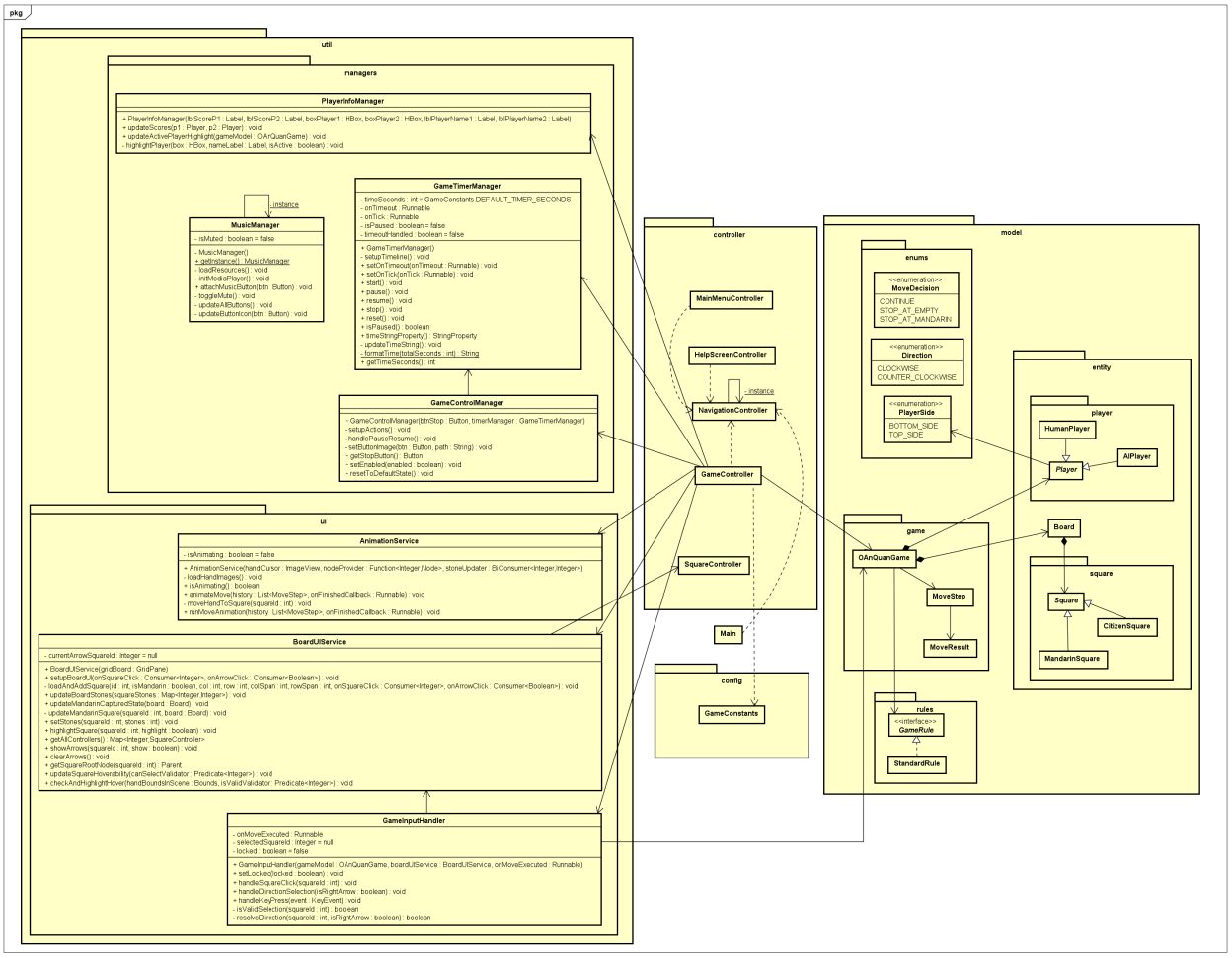
`SquareController`

- `renderVisualStones(int amount)`: show stones based on amount
- `setHoverEnabled(boolean enabled)`: highlighting square when mouse entered

`NavigationController`

- `switchScene(String fxmlPath, String title)`: helper method for switching scenes

Service



AnimationService

- `animateMove(List<MoveStep> history, Runnable onFinishedCallback)` : main method for animating changes, call Runnable when finish
- `moveHandToSquare(int squareId)` : helper method for animating (move hand to a certain square)

BoardUIService

- `setupBoardUI(Consumer<Integer> onSquareClick, Consumer<Boolean> onArrowClick)` : method for creating board UI
- `updateSquareHoverability(Predicate<Integer> canSelectValidator)` : method for updating the highlight ability

GameInputHandler

- `resolveDirection(int squareId, boolean isRightArrow)`: helper method for interpreting arrow direction

- `handleDirectionSelection(boolean isRightArrow)` : method for handling arrow selection input
- `handleSquareClick(int squareId)`: method for handling square clicking

`Manager`

`GameTimerManager`

- `setupTimeline()` : main method for creating timer

`MusicManager`

- `toggleMute()` : method for switching music playing state (on / off)

`PlayerInfoManager`:

- `updateActivePlayerHighlight(OAnQuanGame gameModel)` : update current turn based on the model
- `updateScores(Player p1, Player p2)` : update scores of two players

`GameControlManager`

- `handlePauseResume()` : method for controlling timer state

5. References:

- GUI Idea: https://gamevui.vn/o-an-quan/game
- Standard Rule: from GUI Idea, and many other sources
- MVC Architecture: https://www.youtube.com/watch?v=yfckH7hbCEw